19. Video Streaming (YouTube / Netflix)

> **Interviewer:** Let's design YouTube — or Netflix, pick whichever mental model you like. Creators upload video. Millions of people watch. Design the backend for upload, processing, and playback.

Candidate: Great problem — it's really two systems stapled together: a **write path** (upload → transcode, rare, heavy, tolerant of delay) and a **read path** (stream to viewers, constant, massive, must feel instant). Let me clarify before I commit to anything.

---

1. Clarifying the requirements

> **Candidate:** A few questions: 1. Upload + processing + playback only, or is live streaming in scope too? 2. How big are videos — minutes, or multi-hour? 3. Does quality need to adapt mid-playback as network changes, or is one fixed resolution OK? 4. Global audience — do Tokyo and São Paulo both need fast playback? 5. Do view counts need to be exact and real-time, or "eventually accurate" is fine?
>
> **Interviewer:** On-demand only, no live streaming. Videos range from a minute to a few hours. Yes — adaptive quality switching is required, that's the whole point. Global audience, must feel fast everywhere. View counts can lag by seconds.

Candidate: Then:

Functional - Creator uploads a raw video (large, flaky connections — must be resumable). - System transcodes it into multiple resolutions/bitrates, chunked, asynchronously. - Viewer plays the video, **adapting quality to live bandwidth**, minimal startup delay and buffering. - Track view counts (approximate, near-real-time).

Non-functional — where the hard part is - **Read:write ratio is extreme** — one upload can be watched by millions. The most read-heavy system in this series. - **The bytes are enormous** — gigabytes per video, multiplied by ~5 quality tiers: a petabyte-scale storage problem, not a database problem. - **Global low-latency delivery** — a Mumbai viewer can't round-trip to a US origin per segment. - **Durability** — losing a creator's only copy is unacceptable. - **Startup and rebuffer latency** are the product-defining metrics.

---

2. Estimation — why this is a storage-and-bandwidth problem first

```
Uploads: 500 hours of video uploaded per minute (YouTube-scale, illustrative).
  -> 30,000 hours of raw video every hour.
  avg video ~10 min, raw ~1GB before compression
  -> uploads/day ~= (500*60*24)/10 ~= 72,000 videos/day
  -> raw bytes added/day ~= 72,000 * 1GB = 72 TB/day

After transcoding into ~5 renditions (240p..1080p/4K), total stored bytes per
video roughly DOUBLES to TRIPLES. Hundreds of TB/day, growing forever.
  => video bytes CANNOT live in a database — need object storage built for
     petabyte scale from day one.

Watch traffic dwarfs upload traffic: a viral video can be watched by tens of
millions. A CDN edge near one city might serve hundreds of thousands of
concurrent segment requests at peak — our own servers could never absorb this.

Bandwidth, not storage, is usually the real ceiling:
  1M concurrent viewers * ~3 Mbps average ~= 3 Tbps of egress at peak.
  No single data center's uplink handles that — it MUST be spread across
  thousands of CDN edge locations.
```

Candidate: So two things fall out immediately: video bytes live in object storage, never a database, and the read path is fundamentally a CDN problem, not an application-server problem. Everything else bends around those two facts.

---

3. V1 — the naive design, and where it breaks

```
  [ Creator ] --upload mp4--> [ App Server ] --> [ disk/DB blob, transcode inline ]
  [ Viewer  ] --GET /video/42--> [ App Server ] --> [ streams the single mp4 file ]
```

Why it dies: 1. **Synchronous transcoding blocks the upload request.** A 2-hour 4K video takes minutes to hours to transcode into 5 renditions — the HTTP connection would have to stay open the whole time. 2. **One file, one quality.** A viewer on shaky LTE gets the same file as someone on fiber — either everyone suffers for the slowest viewer, or fast viewers get a poor video. 3. **Every viewer hits our app server directly.** 10M viewers of a viral video means 10M requests fetching multi-GB files against our own servers — instant saturation.

Fix: transcoding becomes **async**, playback becomes **chunked and adaptive-bitrate**, delivery becomes **CDN-first**.

---

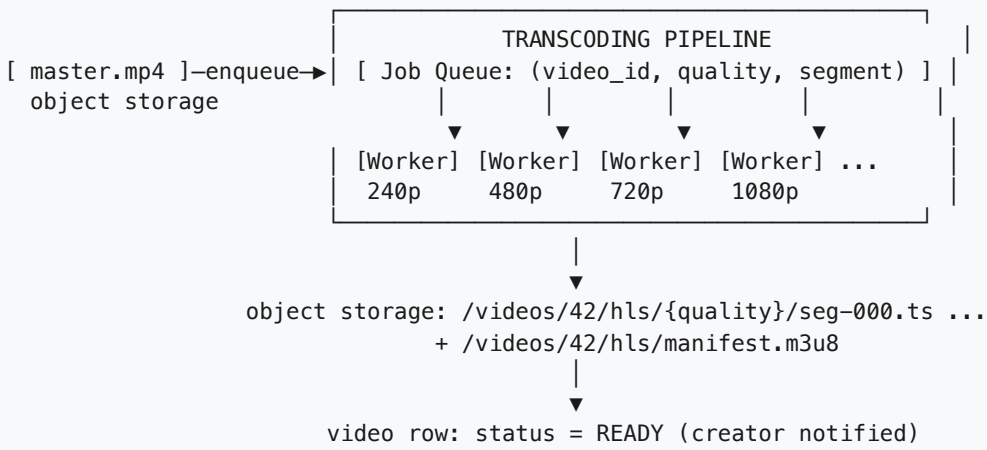
4. The upload + transcoding pipeline

> **Interviewer:** Walk me through what happens the moment a creator hits "upload."

Candidate: Upload is resumable first — mobile networks drop mid-upload constantly.

```
1. POST /uploads            -> server creates video_id + upload_id
2. PUT /uploads/{id}/chunk  -> client uploads in ~5–10MB chunks; server tracks
                               bytes_received so a dropped connection resumes,
                               not restarts
3. POST /uploads/{id}/complete
       -> raw file assembled into object storage: /videos/42/raw/master.mp4
       -> video row: status = PROCESSING
       -> transcode job ENQUEUED (this is where sync would have blocked)
       -> response returns immediately
```

Once the raw master is durable, work happens off the critical path, fanned out across workers:



- **Embarrassingly parallel:** each `(quality, segment)` pair transcodes independently — more workers means linearly faster processing.
- **Idempotent jobs**, keyed by `(video_id, quality, segment_number)`. A crashed worker's job can be retried freely — same output, same key, no double work; we only publish the manifest once all segments for a quality exist.
- **Cost lever:** rarely-watched long-tail uploads can transcode fewer tiers or on-demand; predicted-popular videos get all tiers pre-generated eagerly.

---

5. Chunking for adaptive bitrate

> **Interviewer:** Video is transcoded into multiple qualities. How does the player pick the right one and switch smoothly?

Candidate: This is **HLS/DASH — HTTP-based adaptive streaming.** Each quality is cut into many small chunks (segments), listed in a manifest. The player chooses quality **segment by segment**, not once for the whole video.

```
manifest.m3u8 (tiny text menu, fetched once)
  #QUALITY 240p  (300 kbps) -> seg-000.ts seg-001.ts ...
  #QUALITY 480p  (1  Mbps)  -> seg-000.ts seg-001.ts ...
  #QUALITY 720p  (3  Mbps)  -> seg-000.ts seg-001.ts ...
  #QUALITY 1080p (6  Mbps)  -> seg-000.ts seg-001.ts ...

Playback (each segment ~4-6s of video):
  t=0s     fetch manifest, start at a conservative quality (e.g. 480p)
  t=0-6s   GET .../480p/seg-000.ts, plays while next segment downloads
  t=6-12s  segment arrived early (fast network) -> step UP to 720p
  t=30s    segment arriving late (buffer draining) -> step DOWN to 480p/240p
           -> viewer sees a quality dip, NOT a frozen spinner
```

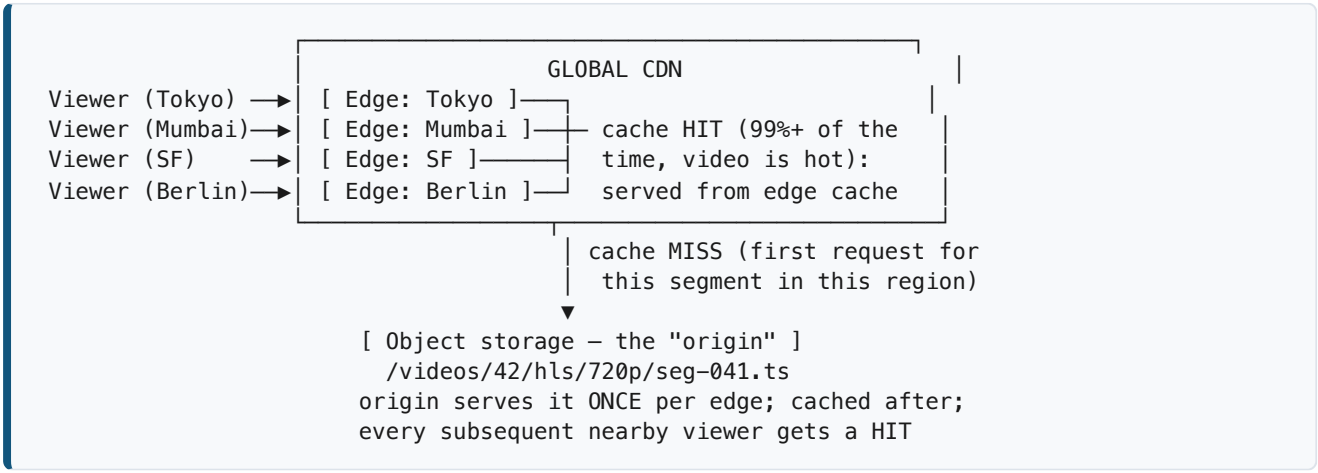
- **Why chunk instead of one file per quality?** A single up-front choice means either a frozen buffer (too high) or permanent blur for someone whose network later improves. Re-deciding every few seconds needs small independent segments.
- **Why plain HTTP GET, not a custom streaming protocol?** Segments are just static files — exactly what CDNs cache out of the box; a custom protocol couldn't be cached by commodity CDN infrastructure and would fight firewalls/proxies. A failed segment fetch just retries that one small file, no session teardown.
- **Segments are immutable** once written — the fact the entire CDN story below leans on.

---

6. CDN delivery — why it dominates this design

> **Interviewer:** Manifest and segments sit in object storage. Millions want to watch. Walk me through the request.

Candidate: This is the crux of the whole system: **our own servers must almost never see a watch request.**



- **Why CDN, not our own load balancer + servers?** Video egress can hit terabits/sec for one popular video. A CDN is a purpose-built, globally distributed cache-plus-delivery network already sitting near every viewer — rebuilding that ourselves means building a whole point-of-presence network, the CDN vendor's entire business.

- **Why it caches so well:** segments are immutable — no invalidation problem, only eviction for space. Best possible case for a cache.
- **Origin only serves the "first fetch" per edge region** — for a viral video that's a handful of origin requests total, not millions.

---

7. Storage tiering — hot vs. cold, and metadata briefly

> **Interviewer:** Petabytes of video, but most isn't watched at any given moment. And what about titles, view counts, recommendations?

Candidate: Watch traffic follows a power law, so tier by heat:

```
HOT  (trending/recent)  -> standard storage class, all renditions, CDN pre-warmed
WARM (occasional views)  -> standard storage, served on cache-miss like normal
COLD (long-tail, most videos by count) -> archival/infrequent-access tier;
     keep raw master + 1-2 mid renditions; transcode others on-demand on
     first request, cache going forward
```

The raw master is always kept, even for cold videos — the one irreplaceable copy everything else can be regenerated from.

Metadata and recommendations, briefly, since storage/delivery is the hard part: - **Metadata** (title, uploader, status, duration) → small relational DB, cached in Redis on read (read far more than written, rarely changes once `READY`). - **View counts** never do per-view row updates (a viral video at 50k views/sec would hammer one row — classic hot-row contention). View events go to a stream; an aggregator batches and flushes one combined increment every few seconds. - **Recommendations** are a separate offline pipeline (watch-history → batch/streaming job → candidate list per user in its own low-latency store) — deliberately decoupled from the video-serving path.

---

8. Technology choices — what and why

Concern	Choice	Why this, and why not the alternative
Video bytes (raw + renditions)	**Object storage (S3-like)**	Built for petabyte scale, cheap per GB, durable via built-in replication, fetched by key with no query engine needed. *Not a database blob column* — a multi-GB row would blow up backup size, replication traffic, and query performance for every other tenant of that DB.
Global delivery to viewers	**CDN (edge cache network)**	Already sits physically close to every viewer; absorbs terabit-scale egress that no origin fleet we run could sustain; segments are immutable so caching has zero invalidation cost. *Not serving from our own origin/LB* — a single viral video would saturate our uplink and servers instantly.

Concern	Choice	Why this, and why not the alternative
Streaming protocol	**HLS/DASH — plain HTTP GET on small segments**	Any CDN caches plain HTTP files for free; survives firewalls/proxies; a failed segment is a cheap retry. *Not a custom TCP streaming protocol* — uncacheable by commodity CDNs, and one dropped packet could kill an entire session instead of one segment.
Transcoding orchestration	**Async job queue + worker pool, keyed by (video_id, quality, segment)**	Embarrassingly parallel work fans out across many workers; idempotent keys make retries free. *Not synchronous inline transcoding* — would block the upload request for minutes-to-hours on long videos.
Metadata	**Relational DB (Postgres/Aurora)**, Redis-cached on read	Small, structured, benefits from real queries ("videos by uploader," "stuck in PROCESSING"); ACID needs are modest but real (status transitions). *Not storing metadata in object storage too* — we'd lose indexed queries and transactional status updates.
View counts	**Stream + in-memory aggregator, batched flush**	Avoids a hot DB row taking 50k writes/sec on a viral video. *Not* `UPDATE ... SET count = count+1` *per view* — that row becomes a global lock and collapses throughput, the same hot-row failure as a naive giveaway counter.
Cold/long-tail storage	**Archival/infrequent-access storage tier**	Most videos, by count, are rarely watched — cheaper tier with on-demand rendition generation trades a one-time latency hit for large ongoing cost savings. *Not keeping everything on hot storage forever* — cost grows unbounded with zero traffic benefit.

Say-it-out-loud justification: "*Object storage holds the actual video bytes because they're too big and too write-once for a database, and a CDN sits in front of it because our own servers could never absorb watch traffic at this scale — the CDN caches aggressively for free because segments are immutable once transcoded. Metadata is small and structured, so it gets a normal relational DB with a Redis read cache; view counts are aggregated off the critical path so a viral video never creates a hot row; and storage tiers by access pattern so we're not paying hot-tier prices for a petabyte of rarely-watched long-tail content.*"

---

9. Multi-region, latency, and consistency

> **Interviewer:** A creator in Berlin uploads; viewers are in Tokyo, São Paulo, New York. Where does everything live, and where do the milliseconds go?

Candidate:

```
UPLOAD:     to the NEAREST regional endpoint (avoids dragging gigabytes across
            an ocean before transcoding even starts). Master lands regionally.
REPLICATE:  object storage replicates master + renditions across regions
            asynchronously, off the upload's critical path.
DISTRIBUTE: CDN edges pull renditions lazily on first local request, or we
            push proactively to edges near predicted demand.
WATCH:      GeoDNS/CDN routing sends each viewer to their nearest edge; even a
            cache miss hits a same-region storage replica, not a cross-ocean origin.
```

Consistency: almost everything here is **eventual**, safely — video bytes are immutable, so a replica lagging by seconds changes nothing about correctness. The one thing that must be **strict**: never publish `status: READY` before every required rendition actually exists — otherwise a viewer gets a manifest pointing at a 404. Opposite profile from a flash-sale-style problem: no scarce counter under contention, just scale and geography.

Startup latency: DNS/TLS to nearest edge (~20-50ms) + cached manifest fetch (~10-30ms) + first segment cache hit (~30-80ms + transfer) — target a few hundred ms, not seconds. Start at a conservative quality rather than gambling on 4K and stalling.

Rebuffer avoidance: the player keeps a 10-30s buffer and compares recent segment download time against playback duration — downloading faster than playing means it's safe to step up; falling behind means step down before the buffer hits zero. The CDN is what makes this signal meaningful — over a distant origin, bandwidth estimates would be dominated by RTT noise instead of real throughput.

---

10. Async and caching — the two levers doing most of the work

> **Interviewer:** Summarize what's async here, and what's cached, and why each is safe.

Async	Why safe off the critical path
Transcoding (all qualities/segments)	Viewer can't watch until `READY` anyway; nobody is waiting synchronously
Cross-region replication	Bytes are immutable — catching up later changes nothing
View-count flush to DB	A few seconds of staleness on a vanity metric is invisible
Creator "processing complete" notice	Not required for playback correctness

```
CACHE — manifest + segments (99%+ of traffic/bytes):
  Player -> CDN edge -> HIT -> return immediately
                     -> MISS -> object storage origin -> cache at edge -> return
CACHE — metadata (tiny):
  GET /videos/{id} -> Redis -> HIT -> return
                            -> MISS -> SQL -> populate Redis
```

Immutability removes the hardest part of caching — invalidation — from the entire video-bytes path. The only "invalidation" that ever happens is an explicit takedown (DMCA/policy), which purges specific CDN paths and flips `status: DELETED` — a rare operation, not a per-write concern.

---

11. Failure modes & graceful degradation

Failure	What happens	Mitigation
Transcode job fails mid-segment	That one `(video, quality, segment)` is missing	Retry just that job — idempotent key, no full-video redo
Upload interrupted (phone loses signal at 60%)	Partial file server-side	Client resumes with the same `upload_id`; only the missing tail is sent
CDN edge miss on a newly-viral video	First requests in that region are slower	One-time origin fetch per edge, cached immediately after — self-heals in seconds
Origin storage node fails	Requests to that node fail	Storage-layer replication serves another copy transparently
Whole region down	Local origin/edge unreachable	CDN reroutes to next-nearest region; cross-region replication keeps the video servable
Metadata DB down	Can't fetch title/status for new sessions	CDN keeps serving cached manifests/segments to in-flight viewers regardless

Design principle: every failure is scoped to one small, independently-retryable piece — one segment, one chunk, one edge, one region — because the system was decomposed that way from the start.

---

12. Follow-up curveballs

> **Interviewer:** A video gets taken down for a policy violation, cached on thousands of CDN edges worldwide — how fast can it actually disappear?

Candidate: Flip `status: DELETED` immediately (manifest 404s, no new session can start), then issue a **CDN purge** for that video's paths — propagates within seconds to low-minutes on most CDNs. Anyone already mid-playback with segments cached locally might finish that session; for true "everywhere, now" we'd pay for the CDN's fast-purge tier over TTL expiry.

> **Interviewer:** How would live streaming change this design?

Candidate: No pre-existing file to transcode — segments must be produced and pushed to CDN edges within seconds of filming, turning section 4's pipeline into a real-time loop with a much tighter latency budget, and the manifest becomes a live, growing list. Viewer-side ABR logic barely changes.

> **Interviewer:** How do you decide which resolutions to generate, instead of always doing all five?

Candidate: Cost-driven, tied to predicted popularity — likely-viral uploads get all tiers pre-generated and pre-warmed; long-tail uploads get one or two tiers eagerly and the rest transcoded lazily on first request, paying that latency only if someone actually asks.

> **Interviewer:** How does DRM fit in without redesigning everything?

Candidate: It layers on top — segments are encrypted at transcode time, and the player fetches a short-lived decryption license per session before decoding. Chunking, manifests, and CDN caching are unchanged; CDNs happily cache encrypted bytes since decryption is client-side.

---

13. Deep-dive: "Why not skip the CDN and just serve straight from the object store — and why not transcode on the fly instead of pre-baking a whole ABR ladder?"

> **Interviewer:** Object storage can serve HTTP GETs directly, and it's already durable and replicated. Why bolt a whole CDN in front of it? And separately — why pre-transcode five renditions of every video up front? Why not just transcode to whatever bitrate the viewer asks for, on demand, and skip storing the ones nobody watches?

Candidate: Both are the same mistake at different layers: **doing expensive work at request time, on the hot path, for traffic that's fundamentally repeatable.** Let me take them one at a time.

Why not serve straight from the origin/object store?

Three problems compound:

1. **Bandwidth, not just cost.** Object storage gives you durability and *a* HTTP endpoint, but it sits in a handful of regions. A CDN gives you thousands of points of presence already peered with residential ISPs. Without it, every one of a viral video's 10M viewers drags bytes across the same few uplinks — that's the 3 Tbps number from §2, aimed at infrastructure never built to be an edge network.
2. **Global latency.** A Mumbai viewer round-tripping to a US-region object store pays 150-250ms **per segment request**, every 4-6 seconds, for the whole video. That's the difference between a "few hundred ms" startup and a visibly laggy player.
3. **Cost shape.** Object storage egress is billed per GB and isn't discounted for serving the same byte range a million times. A CDN's entire business model is amortizing one origin fetch across massive repeat reads — which is exactly our access pattern, since segments are immutable and rewatched constantly.

None of this means object storage is wrong — it's still the *origin*, the durable source of truth. The point is that **origin and delivery are different jobs**, and only a CDN is built for the second one.

Why not transcode on the fly instead of pre-baking the ABR ladder?

Same shape of mistake, moved to compute instead of network:

1. **Transcoding is CPU/GPU-heavy and slow.** Encoding even one rendition of a few minutes of video takes real wall-clock time — seconds to minutes depending on resolution and codec. Doing

it *while a viewer waits for their first segment* turns a sub-second startup into a multi-second (or worse) stall, for every viewer, every time, because "on the fly" means *no caching of the work itself*.
2. **It doesn't amortize.** A pre-transcoded segment is encoded once and read by millions of viewers off a cache. An on-the-fly transcode redoes the same CPU work **per request** (or you'd have to cache the output anyway — at which point you've just reinvented pre-transcoding, badly, on the critical path instead of ahead of time).
3. **ABR needs the ladder to already exist.** Adaptive bitrate switching (§5) works by the player picking a *different pre-existing* rendition segment-by-segment as bandwidth changes. If rendition N doesn't exist yet, "switch to 480p" means "wait several seconds while we encode 480p" — the opposite of graceful degradation.

The one place on-the-fly transcoding earns its keep is the **cold/long-tail case from §7**: rarely-watched videos where paying a one-time latency hit on first request is cheaper than eagerly generating and storing five renditions nobody may ever watch. That's a deliberate, narrow exception — not the default.

Concern	Origin/object store direct	CDN in front
Egress at viral scale	Saturates our own uplinks; billed per-GB with no benefit from repeat reads	Thousands of edges absorb terabit-scale egress; one fetch amortized across millions of viewers
Global latency	Every viewer pays RTT to a handful of regions	Viewer hits a nearby edge; origin RTT paid once per region, not per viewer
Cache invalidation cost	N/A — no cache	Zero, because segments are immutable (§6)
Failure blast radius	Origin outage = every viewer affected	Origin outage = only cache-miss traffic affected; edges keep serving hot content

Concern	On-the-fly transcode per request	Pre-transcoded ABR ladder
Latency to first byte	Seconds-to-minutes of encode time in the critical path	Milliseconds — just a cache/storage read
Compute cost at scale	Repeats the same encode for every request unless you cache it (which just becomes pre-transcoding)	Paid once per `(video, quality, segment)`, amortized over every future view
ABR quality-switching	Can't switch to a rendition that doesn't exist yet	Player switches segment-by-segment among renditions that already exist
Best fit	Cold/long-tail videos, rarely watched, latency hit acceptable once	Hot/warm videos — the overwhelming majority of watch traffic

> **Rule of thumb:** *If the same expensive work (a network fetch or a transcode) will be repeated for many viewers, do it once, ahead of the request, and cache/store the result. Pay per-request cost only where the "many viewers" assumption actually breaks down — the long-tail cold path — and even then, cache the first result so the second viewer doesn't pay again.*

---

14. Deep-dive: the cache-miss storm on a hot launch, and ABR under fluctuating bandwidth

> **Interviewer:** A massively hyped video — a new episode drop, a huge live event replay — goes live at a scheduled instant. Millions of people refresh and hit play within the same few seconds. It's brand new, so **no CDN edge has it cached anywhere**. Walk me through what happens, and how you'd stop it from falling over.

Candidate: This is the CDN's version of the giveaway's thundering herd (§3 in the burger doc) — except instead of one hot database row, it's **one origin fetch path** that every edge in the world suddenly needs at once.

Why it happens

```
t=0s: video goes live. manifest.m3u8 + all segments exist ONLY in origin
      object storage. No edge anywhere has a cached copy yet.

t=0–2s: millions of players fetch the manifest, then request seg-000.ts
        at their starting quality.

          Tokyo edge    ──┐
          Mumbai edge   ──┤  ALL see: cache MISS (first request ever
          SF edge       ──┤  for this exact segment/quality at this edge)
          São Paulo edge──┘
                   │
                   ▼  every miss falls through to the SAME origin
       ┌──────────────────────────────────────────┐
       │        OBJECT STORAGE ORIGIN             │
       │  seg-000.ts requested simultaneously by  │
       │  hundreds of edges, each on behalf of    │
       │  thousands of viewers behind it          │
       └──────────────────────────────────────────┘
                   │
                   ▼
       origin uplink saturates -> latency spikes -> some
       fetches time out -> edges retry -> MORE origin load
       -> classic positive-feedback stampede
```

The cruel part: this isn't spread evenly. Everyone wants the **same few opening segments** of the **same video** at the **same instant** — it's a hot-key problem, just like the burger counter, except the "key" is `(video_id, quality, segment_0)` and the contention point is the origin's network capacity, not a database lock.

Fix 1 — origin shield / tiered (hierarchical) caching

Don't let every edge hit the origin independently. Insert a **mid-tier ("shield") cache** — one layer of regional caches that sit between edges and origin. Edges miss to their regional shield, not straight to origin; only the *shield's* miss reaches the origin, and only once per region:

```
 1000s of edges  ──miss──▶  ~10–20 regional shield caches  ──miss──▶  origin
 (fan-in reduced from "thousands of independent fetches" to "tens")
```

This alone turns "one fetch per edge" into "one fetch per shield region," cutting origin load by orders of magnitude. Most commercial CDNs (CloudFront, Akamai, Fastly) offer this natively.

Fix 2 — request collapsing (coalescing) at each cache tier

Within a single edge or shield node, if 10,000 concurrent viewers behind it all miss on the exact same segment within the same instant, don't issue 10,000 upstream requests. **Collapse** them: the first request triggers the fetch, the other 9,999 wait on that in-flight fetch and all get served the same response once it lands.

```
edge node, segment X, t=0:
  req 1  ──► MISS ──► fetch upstream (in flight)
  req 2..10000  ──► MISS ──► "someone's already fetching this" ──► wait
  fetch completes ──► all 10,000 served from the one fetched copy
```

This is the single highest-leverage fix for the exact failure mode described — it directly caps "N viewers behind one edge" from becoming "N origin requests."

Fix 3 — pre-warming / pre-positioning for known launches

For a *scheduled* drop (unlike an organically-viral video), we know the spike is coming. Push the manifest and at least the opening segments of each rendition to edges/shields **before** t=0, instead of waiting for organic cache misses to populate them:

```
t = -30min: publish is done, but CDN pre-warm job proactively
            PUSHES seg-000..seg-010 of every quality to major
            regional edges ahead of the scheduled release time
t = 0:      "misses" barely happen — the popular opening segments
            are already sitting at the edge when the crowd arrives
```

This turns a cold-cache stampede into a warm-cache non-event for the part of the catalog we can predict — the exact lever mentioned in §4's "predicted-popular videos get all tiers pre- generated eagerly," extended from transcoding to CDN placement.

Fix 4 — per-chunk (per-segment) caching, not per-video

Because HLS/DASH already chunks video into small immutable segments (§5), the "cache unit" is tiny and independent. This matters here specifically because it bounds the blast radius: a miss storm on `seg-000` doesn't imply a miss storm on the 2-hour video's `seg-500` — demand concentrates on whatever's currently playing, and each segment's stampede is resolved independently (and briefly) by fixes 1-2. If we cached whole-video files instead, the "unit of contention" would be gigabytes instead of a few hundred KB, and collapsing/shielding would be far less effective.

Recommended approach

Layer all four, but the priority order for engineering effort is: 1. **Origin shield / tiered caching** — always on, protects the origin unconditionally, regardless of whether a launch was predicted. 2. **Request collapsing** — always on, essentially free, directly targets "millions of viewers, same segment, same instant." 3. **Pre-warming** — highest leverage specifically for *known* scheduled launches; requires the business to tell engineering "this drops at 9am," same signal used for eager transcoding. 4. **Per-chunk**

caching — already a side-effect of choosing HLS/DASH; nothing extra to build, just confirms the segment size is small enough to keep contention windows short.

Adaptive bitrate under fluctuating bandwidth — the other half of "hard edge case"

The stampede above is about *origin* capacity. The complementary hard case is a single viewer's bandwidth **fluctuating mid-playback** — a subway train going through tunnels, a wifi handoff — and ABR has to react without either stalling or oscillating:

```
buffer health (seconds of playable video already downloaded) drives the decision,
not raw bandwidth alone:

t=10s  buffer=25s, last segment arrived in 1.5s for a 6s segment (4x margin)
       -> safe to step UP: 480p -> 720p
t=16s  network degrades; segment takes 5.5s to arrive for a 6s segment
       -> buffer barely growing -> STAY, don't step up again yet
t=22s  segment takes 9s to arrive (network still bad) -> buffer draining
       -> step DOWN immediately: 720p -> 480p -> 240p if it keeps draining
t=30s  buffer hits ~2s -> emergency: drop to lowest rendition to avoid a
       full stall; a visible quality dip beats a frozen spinner
```

The failure mode to guard against is **oscillation** — flipping quality every few seconds because a single slow segment triggered a downgrade and the very next fast one triggered an upgrade. Fix: asymmetric thresholds (downgrade fast/eagerly, upgrade only after several consecutive healthy segments) and basing the decision on **buffer trend**, not single-sample throughput — exactly why this only works well when segments are being served from a nearby, low-jitter CDN edge (§9) rather than a distant origin whose RTT noise would dominate the signal.

> **What to say out loud:** "*A cache-miss storm on a hot launch is the CDN's version of a thundering herd — the fix is the same shape as the burger counter's sharding: don't let every caller hit the same contended resource independently. Origin shielding fans thousands of edge misses down to a handful of regional requests, request collapsing turns N concurrent viewers' misses into one upstream fetch, and pre-warming eliminates the miss entirely for launches we can predict. Per-chunk caching is what makes all of this cheap — the unit of contention is a few hundred KB, not a multi-gigabyte file. Adaptive bitrate is a separate but related problem — it's about one viewer's fluctuating bandwidth, not global origin load — and it's solved by reacting to buffer-drain trend with asymmetric up/down thresholds, which only produces a clean signal because the CDN keeps segment RTT low and non-noisy in the first place.*"

Summary — the mental model

Video streaming is two systems bolted together with wildly different traffic profiles: a rare, heavy, latency-tolerant **write path** (upload → async, parallel, idempotent transcoding into chunked adaptive-bitrate renditions) and a constant, massive, latency-intolerant **read path** that must almost never touch our own servers. Everything bends around one fact: **video bytes are immutable once produced**, which is what lets a **CDN** cache them aggressively with zero invalidation cost, turning a multi-terabit delivery problem into "our origin serves each segment roughly once per edge region." Object storage — not a database —

holds the bytes; a small relational DB holds metadata; view counts aggregate asynchronously to dodge hot-row contention; and storage tiers by access pattern to keep petabyte-scale costs sane. If you can explain why the CDN, not the database or the app server, is the single most important component here, you've understood the problem.